NAVlib Manual

Introduction

NAVlib is a .NET library and uses .NET Framework 4.6.1. It can be used from any language which supports the .NET platform. The NAVlib API has been developed by OxTS to allow communication with OxTS products and processing of stream data both from live devices and recorded data files. It consists of 2 main libraries:

• **OxTS.NavLib.LowLevel** - This is the basic link to the low-level C++ functionality. It is not recommended to use this dll directly.

• **OxTS.NavLib.Core** – The main functionality of NAVlib including the datastore manager classes.

There are also extension libraries available such as **OxTS.Navlib.PluginInterface** required to create plugins for OxTS applications.

NAVlib is made up of many interlocking parts, the main entry points for most application will be the data store manager classes (**RealTimeDataStoreManager** and **FileDataStoreManager**). These classes provide the functionality for most applications of NAVlib. Realtime is used for streams arriving from live devices and file replays. File is used for opening previously logged files (.xcom, .ncom etc). All NAVlib classes and methods are contained within the OxTS.NavLib namespace.

Streams and Transports

NAVlib uses the concept of streams and transports to represent measurement data and devices on the network.

A stream is defined as a container for measurements of a certain type, each stream type contains different measurements.

A transport is defined as a container which can contain one or more streams and is used to transport the streams over a network.

The stream types are:

Stream	Description
BCOM	Stream type used for the RT-Base S
CAN	Stream sent over CAN bus
MCOM	Maritime navigation stream
NCOM	Standard navigation stream
RCOM	Stream used on the Range system
UCOM	Universal stream type

The transport types are:

Transport	Streams Available	Receive Port	Send Port
BCOM	BCOM	50470	3001
Nav	NCOM, MCOM	3000	3001
Rng	RCOM, NCOM	3003	3004
Virtual	BCOM, MCOM, NCOM, RCOM, CAN	N/A	N/A
XCOM	All current stream types	50475	3001

The receive port refers to the UDP port opened by NAVlib to receive live data from devices on the network. The send port is used when sending commands to a device on the network. Those ports must be open for NAVlib to function correctly.

Ids

Within NAVlib you will see many different types of Ids related to streams, the most important of these Ids are the StreamId and ResourceId:

• StreamId

• ResourceId/FileId

Note: Ids are not shared between the realtime and file datastore.

Stream Ids

Within NAVlib each stream is given a StreamId that is used to uniquely identify the stream. If a stream times out (no data is received for 10 seconds), the stream is classified as “Dead” (IsAlive == false). When a device stream timeouts then later sends the same stream, this new stream will not have the same StreamId, the user must match the stream to the one used previous manually. This can be done using the OriginalStreamId property.

Resource Ids

ResourceId or FileId (in the file datastore) is a unique for groups of streams from the same device using the same transport type or the same file added to the data store. Multiple streams can share a ResourceId.

Stream Items

Stream items are how stream information is represented within NAVlib. The realtime data store and file data store each have their own stream item type (**RealTimeStreamItem** and **FileStreamItem** respectively). These stream items contain information about the stream including stream type, transport type and Ids. The current streams can be retrieved from the data store managers.

NAVlib also supports the concept of virtual streams, a virtual stream is created entirely within the software and does not transmit any measurement data. By default, these streams are not available. To start them use the **StartVirtual** method (currently only available in realtime), these streams will then automatically be returned when requesting streams. To disable them the **HideVirtualStreams** method can be used.

Realtime Datastore Manager

The RealTimeDataStoreManager class is used to retrieve data from streams being broadcast over the network by live devices. It also has the functionality to replay files, the streams in these files will appear as if they are live data. There are a few ways of retrieving data from these streams using NAVlib. These methods require using ConfigureMeasurementsToRead to tell the datastore what measurements we are interested in receiving, the type of data returned depends on the argument “measurement_type”:

- **Single** is used to request a single measurement.
- **Multiple** is used to request many measurements.
- **MultipleStreamed** is used to get historical data and will return an historical list for each measurement requested given a timescale. (Note: there is a lmit on the number of measurements returned, it is assumed this function will be called more than once per second) Configuring the measurements will return a CallerId, this CallerId is what will be used to retrieve the specific data.

To retrieve the data, use the GetRealTimeData method with the unique caller Id that the measurements were configured with. This will return a RealTimeDataBase type, from this class we can use the GetData or GetStreamData methods to get the actual measurement data. See the NAVlib examples projects (Get_Direct_Measurements and GetHistoryData) for example code showing how this is done.

The realtime datastore manager also allows commands to be sent directly to any device found on the network using the SendCommandToDevice function, note that this takes ResourceId not StreamId as an argument as the resource more accurately represents the device.

File Datastore Manager

The file datastore manager is used to load data from files containing previously logged data. It has two ways of retrieving data from streams. These are known as retain and direct.

- **Retain** is intended to be used when parsing the whole file or portions of the file. It uses the same ConfigureMeasurementsToRead function as found in the realtime datastore manager. The data must also be decoded with a given increment before using the retain method, this can be done using the DecodeData method. After decoding measurement data can be retrieved using the GetStreamDataForInstant or GetStreamDataForPeriod methods.
- **Direct** can be used to get data at a specific time in the file, this is useful when we want random access to the data within the file and can be faster than having to decode the entire file. When using direct we do not need to configure the measurements or decode the data. When using direct decode, the BeginDirectDecode and EndDirectDecode methods should be used, this allows the data store to seek more efficiently. The functions which use the direct method have direct in their name (e.g. GetDirectStreamDataForInstant and GetDirectStreamDataForPeriod).

Note: When decoding a file, there is the possibility of using too much memory if the increment is too low and number of measurements too high. This can be a reason to use the direct method as this does not decode the whole file at once.

Measurements

Each stream type contains different measurements. The measurement categorisation class provides access to the available measurements for each stream type. It is available from the data store manager classes, the datastore manager class also provides a function (GetMeasurementsFromStream), that can get the measurements available for any stream already connected, this must be used when dealing with CAN streams.

The MeasurementCategories class provides multiple ways to get the measurements available for a particular stream type.For example:

- **GetAllNavMeasurements** retrieves all measurements available in Nav streams.
- **GetAllNavMeasurementsForCategory** retrieves measurements by category (Time, Velocity, Acceleration etc). The categories are available using the GetNavCategories function.
- **GetGraphableNavMeasurementsInCategory** gets all measurements that are marked as Graphable.

The examples shown here are for the Nav stream type, but similar functions are available for other stream types.

Each stream type has its own categorisation type (NavMeasurementCategorization, BCOMMeasurementCategorization etc) that the measurement categories class uses to return the measurements available for that stream, these classes also return other

information pertinent to the measurement such as the description, output format and units.

Plugins

Plugins can be created for NAVdisplay using the **INAVdisplayPlugin** interface available in the OxTS.Navlib.PluginInterface dll. To create a plugin you should create dll with a class that inherit the interface and implement the functionality including entering your license key in the **GetNAVlibLicense** function. The inheriting class must also provide an export of the same interface:

[Export(typeof(INAVdisplayPlugin))]

The plugin interface includes a **ConnectToDataStore** function. This function is used by the launching application to give the plugin a reference to a **RealTimeDataStoreManagerAsync** object. This class contains most of the functionality of the **RealTimeDataStoreManager** but uses an async interface.

To run your plugin inside NAVdisplay you must copy the dll and any externally referenced dlls into the NAVdisplay plugins folder (C:\Program Files (x86)\OxTS\NAVdisplay\plugins).

An example of both a Winforms and WPF plugin is available in the example solution included in the installer. Note: Plugins created this way are only available from NAvsuite 2.1 onwards.

Further Documentation

- Included in the installer is a full reference document containing documentation for all classes and functions in NAVlib. If using Microsoft Visual Studio this documentation is also available through the Intellisense. The OxTS.NAVlib.Internal namespace is intentionally left undocumented and users are not expected to use these classes.
- A Quickstart guide is available for getting setup using NAVlib and creating a simple application using the realtime data store manager.
- Example applications are included, showing both the functionality of the realtime data store manager and file data store manager.
- The examples can also be accessed on GitHub by clicking here.
- Documentation containing information for every measurement for available in NAVlib has also been included. A separate document is available for BCOM, Nav and RCOM streams